



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)

КАФЕДРА «Информационная безопасность» (ИУ8)

ТЕОРИЯ ИГР И ИССЛЕДОВАНИЕ ОПЕРАЦИЙ

Лабораторная работа № 1

на тему:

«Решение матричной игры в смешанных стратегиях
аналитическим методом и методом Брауна-Робинсон»

Вариант 13

Студент:

ИУ8-105
(Группа)

Б. Е. Шекунов
(И. О. Фамилия)

Преподаватель:

Н.С. Коннова
(И.О. Фамилия)

Москва, 2026 г.

Цель работы:

Изучить аналитический (обратной матрицы) и численный (Брауна-Робинсон) методы нахождения смешанных стратегий в антагонистической игре двух лиц в нормальной форме.

Постановка задачи:

Найти цену игры и оптимальные стратегии обоих игроков методами обратной матрицы и Брауна-Робинсон. Сравнить полученные результаты.

Исходные данные:

Исследуется матричная игра с платёжной матрицей игрока А

$$C = \begin{pmatrix} 9 & 10 & 13 \\ 1 & 18 & 11 \\ 17 & 4 & 0 \end{pmatrix}$$

Ход работы:

Найдём нижнюю и верхнюю цены игры.

Минимальные элементы по строкам:

$$\min(9, 10, 13) = 9, \min(1, 18, 11) = 1, \min(17, 4, 0) = 0.$$

Следовательно, нижняя цена игры равна

$$\underline{v} = \max(9, 1, 0) = 9.$$

Максимальные элементы по столбцам:

$$\max(9, 1, 17) = 17, \max(10, 18, 4) = 18, \max(13, 11, 0) = 13.$$

Следовательно, верхняя цена игры равна

$$\bar{v} = \min(17, 18, 13) = 13.$$

Так как $\underline{v} = 9 \neq 13 = \bar{v}$, седловая точка отсутствует, поэтому игра решается в смешанных стратегиях.

Решение аналитическим методом

Так как матрица игры квадратная и невырожденная, используем формулы обратной матрицы:

$$x^* = \frac{uC^{-1}}{uC^{-1}u^T}; \quad y^* = \frac{C^{-1}u^T}{uC^{-1}u^T}; \quad v = \frac{1}{uC^{-1}u^T}$$

где $u = (1 \ 1 \ 1)$.

Для данной матрицы получена обратная матрица:

$$C^{-1} = \begin{pmatrix} \frac{11}{613} & -\frac{13}{613} & \frac{31}{613} \\ -\frac{187}{2425} & \frac{221}{2425} & \frac{43}{1226} \\ \frac{151}{1226} & -\frac{67}{1226} & -\frac{38}{613} \end{pmatrix}$$

Тогда

$$uC^{-1} = \begin{pmatrix} \frac{159}{2425} & \frac{35}{2425} & \frac{29}{1226} \end{pmatrix}$$

$$C^{-1}u^T = \begin{pmatrix} \frac{29}{613} \\ \frac{30}{613} \\ \frac{4}{613} \end{pmatrix}$$

$$uC^{-1}u^T = \frac{63}{613}$$

Отсюда цена игры:

$$v = \frac{1}{uC^{-1}u^T} = \frac{613}{63} \approx 9,730159$$

Оптимальная смешанная стратегия игрока А:

$$x^* = \left(\frac{53}{84} \quad \frac{5}{36} \quad \frac{29}{126} \right) \approx (0,630952; 0,138889; 0,230159).$$

Оптимальная смешанная стратегия игрока В:

$$y^* = \left(\frac{29}{63} \quad \frac{10}{21} \quad \frac{4}{63} \right) \approx (0,460317; 0,476190; 0,063492).$$

Все вероятности в смешанных стратегиях обоих игроков неотрицательные и в сумме дают единицу. Кроме того, проверка показала, что ожидаемые выигрыши по всем столбцам при стратегии x^* равны цене игры:

$$x^*C = (9,730159; 9,730159; 9,730159).$$

Аналогично ожидаемые выигрыши по всем строкам при стратегии y^* также равны цене игры:

$$Cy^* = \begin{pmatrix} 9,730159 \\ 9,730159 \\ 9,730159 \end{pmatrix}$$

Следовательно, найденные стратегии являются оптимальными, а цена игры равна

$$v = 9,730159$$

Решение методом Брауна-Робинсон

Для численного решения использован метод Брауна-Робинсон.

Начальная пара стратегий: x_1, y_1 .

Требуемая точность: $\varepsilon \leq 0,1$.

На каждой итерации:

- игрок А выбирает стратегию, соответствующую максимальному накопленному выигрышу;
- игрок В выбирает стратегию, соответствующую минимальному накопленному проигрышу.

Требуемая точность достигнута на 710-й итерации

$$\varepsilon = 0,099713 \leq 0,1.$$

Так как полная таблица имеет слишком много строк, целесообразно привести начальные и конечные шаги алгоритма.

Таблица 1 - Таблица итераций метода Брауна-Робинсон

k	выбор А	выбор В	накопленные выигрыши А	накопленные проигрыши В	v_{max}/k	v_{min}/k	E
1	x1	y1	[9, 1, 17]	[9, 10, 13]	17.000000	9.000000	8.000000
2	x3	y1	[18, 2, 34]	[26, 14, 13]	17.000000	6.500000	8.000000
3	x3	y3	[31, 13, 34]	[43, 18, 13]	11.333333	4.333333	2.333333
4	x3	y3	[44, 24, 34]	[60, 22, 13]	11.000000	3.250000	2.000000
5	x1	y3	[57, 35, 34]	[69, 32, 26]	11.400000	5.200000	2.000000
6	x1	y3	[70, 46, 34]	[78, 42, 39]	11.666667	6.500000	2.000000
7	x1	y3	[83, 57, 34]	[87, 52, 52]	11.857143	7.428571	2.000000
...	...	...	...	...	...	...	...
704	x2	y1	[6913, 6917, 6637]	[6712, 6918, 7341]	9.825284	9.534091	0.104497
705	x2	y1	[6922, 6918, 6654]	[6713, 6936, 7352]	9.818440	9.521986	0.104497
706	x1	y1	[6931, 6919, 6671]	[6722, 6946, 7365]	9.817280	9.521246	0.104317
707	x1	y1	[6940, 6920, 6688]	[6731, 6956, 7378]	9.816124	9.520509	0.103162
708	x1	y1	[6949, 6921, 6705]	[6740, 6966, 7391]	9.814972	9.519774	0.102009
709	x1	y1	[6958, 6922, 6722]	[6749, 6976, 7404]	9.813822	9.519041	0.100859
710	x1	y1	[6967, 6923, 6739]	[6758, 6986, 7417]	9.812676	9.518310	0.099713

Лучшая верхняя оценка цены игры:

$$\bar{v} = 9,812676.$$

Лучшая нижняя оценка цены игры:

$$v = 9,712963.$$

Следовательно,

$$9,712963 \leq v \leq 9,812676.$$

Приближённая цена игры определяется как середина этого интервала:

$$\tilde{v} = \frac{9,812676 + 9,712963}{2} = 9,762820.$$

Дальше необходимо получить приближённые смешанные стратегии.

Число использований стратегий игроками за 710 итераций:

$$N_A = (502, 81, 127), N_B = (319, 329, 62).$$

Отсюда приближённые смешанные стратегии:

$$\tilde{x} = \left(\frac{502}{710} \quad \frac{81}{710} \quad \frac{127}{710} \right) \approx (0,707042; 0,114085; 0,178873),$$

$$\tilde{y} = \left(\frac{319}{710} \quad \frac{329}{710} \quad \frac{62}{710} \right) \approx (0,449296; 0,463380; 0,087324).$$

Сравнение аналитического и численного решений:

Аналитическое решение дало:

$$v = 9,730159,$$

$$x^* = (0,630952; 0,138889; 0,230159),$$

$$y^* = (0,460317; 0,476190; 0,063492).$$

Метод Брауна-Робинсон дал:

$$\tilde{v} = 9,762820,$$

$$\tilde{x} = (0,707042; 0,114085; 0,178873),$$

$$\tilde{y} = (0,449296; 0,463380; 0,087324).$$

Абсолютное отклонение приближённой цены игры от аналитической составило

$$|\tilde{v} - v| = |9,762820 - 9,730159| = 0,032661.$$

Это отклонение меньше достигнутой оценки погрешности итерационного метода, что подтверждает корректность вычислений.

Отклонения по стратегиям также невелики. Наиболее заметное расхождение наблюдается по первой и третьей стратегиям игрока А, что объясняется конечным числом итераций. Для игрока В приближённая стратегия оказалась ближе к аналитической.

Так же для наглядности можно привести графики.

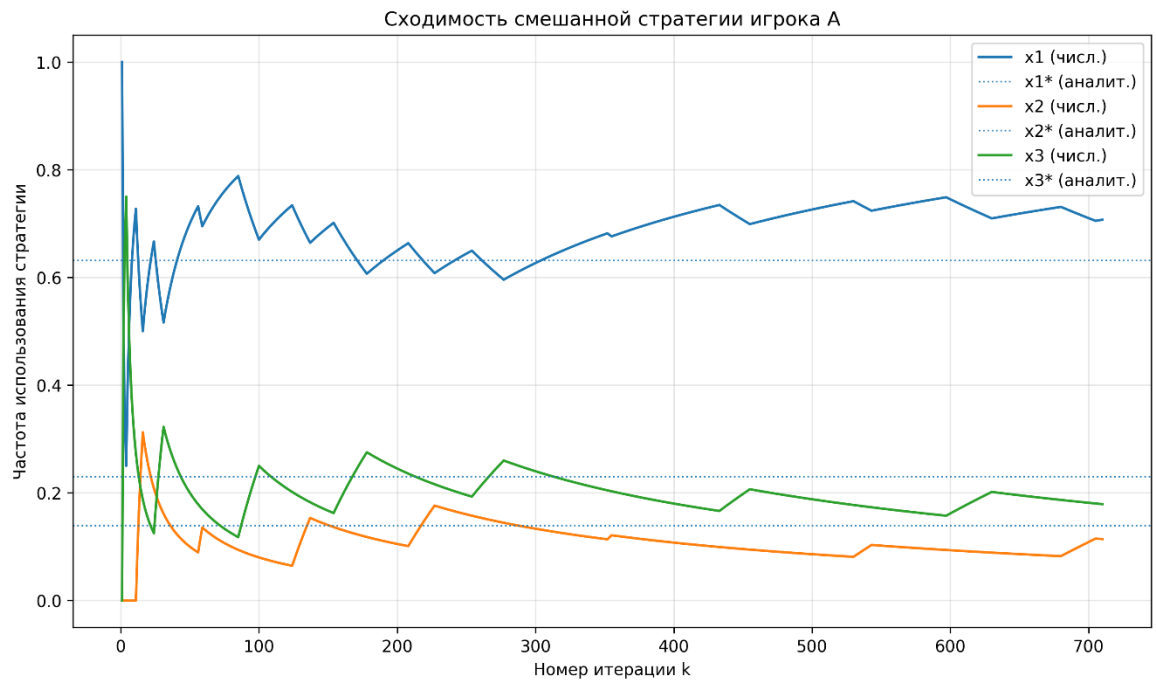


Рисунок 1 – Сходимость смешанной стратегии игрока А

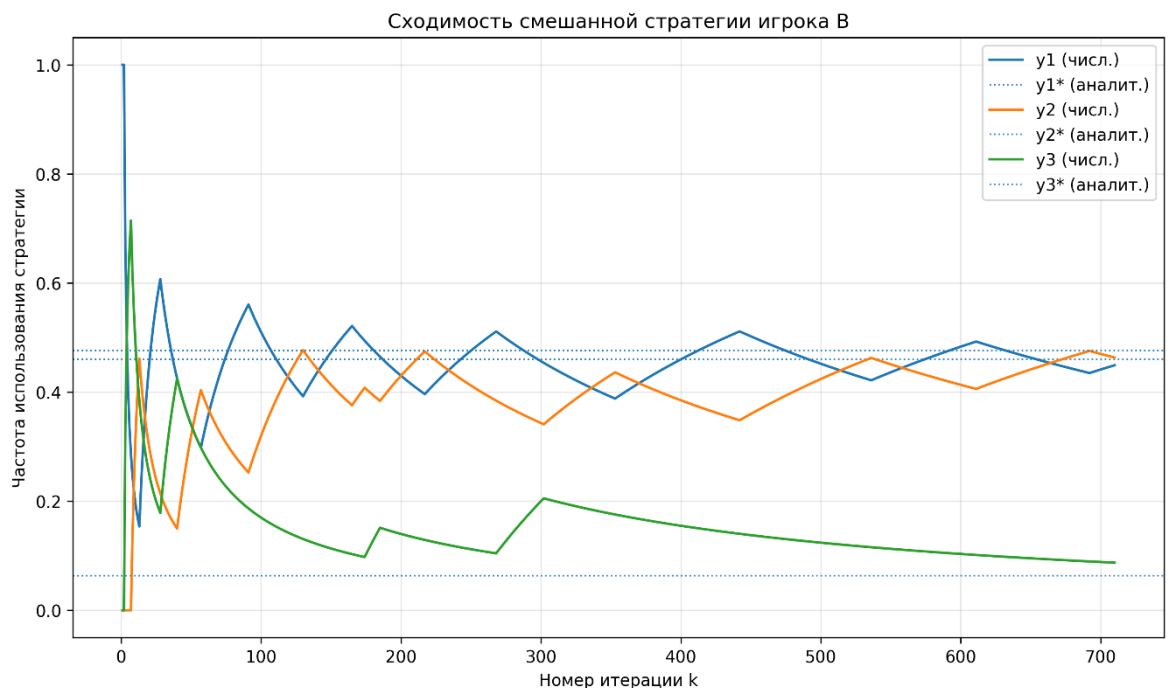


Рисунок 2 – Сходимость смешанной стратегии игрока В

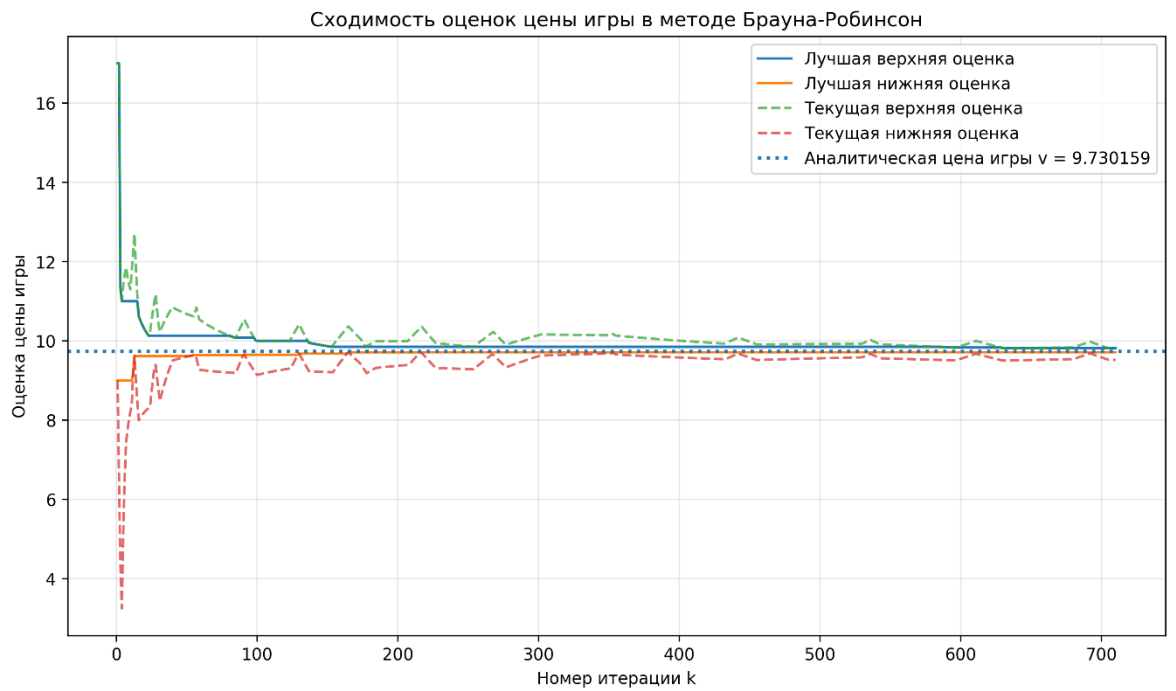


Рисунок 3 – Сходимость оценок цены игры в методе Брауна-робинсон

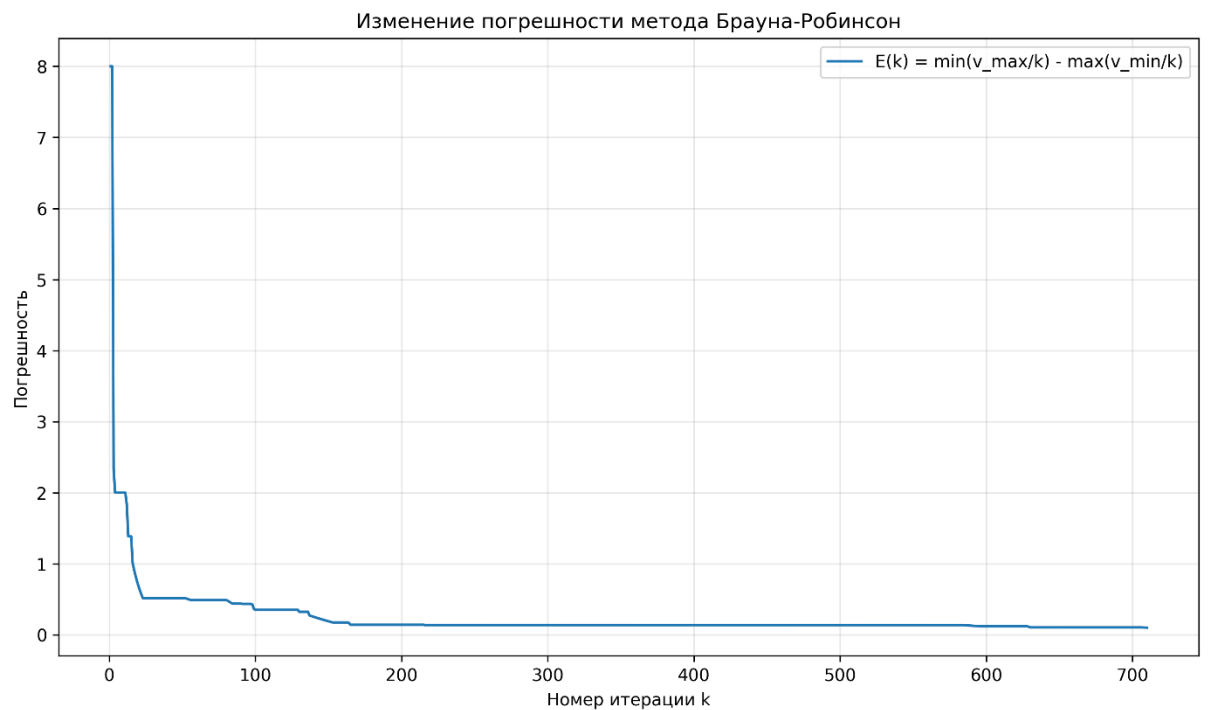


Рисунок 4 – Изменение погрешности в методе Брауна-робинсон

По полученным данным видно, что с ростом числа итераций частоты использования стратегий стабилизируются около значений аналитического решения, верхняя и нижняя оценки цены игры сближаются, а погрешность уменьшается.

Вывод:

В работе была исследована матричная игра с платёжной матрицей

$$C = \begin{pmatrix} 9 & 10 & 13 \\ 1 & 18 & 11 \\ 17 & 4 & 0 \end{pmatrix}$$

В ходе анализа установлено, что седловая точка отсутствует, так как нижняя и верхняя цены игры не совпадают:

$$\underline{v} = 9, \bar{v} = 13.$$

Аналитическим методом найдено точное решение игры:

$$v = \frac{613}{63} \approx 9,730159,$$

$$x^* = (0,630952; 0,138889; 0,230159),$$

$$y^* = (0,460317; 0,476190; 0,063492).$$

Численным методом Брауна—Робинсона за 710 итераций получено приближённое решение с погрешностью:

$$E = 0,099713 \leq 0,1$$

При этом приближённая цена игры составила

$$\tilde{v} = 9,762820,$$

а приближённые смешанные стратегии:

$$\tilde{x} = (0,707042; 0,114085; 0,178873),$$

$$\tilde{y} = (0,449296; 0,463380; 0,087324).$$

Сравнение результатов показало, что численный метод даёт достаточно точное приближение аналитического решения. Следовательно, поставленная в лабораторной работе задача решена полностью.

Приложение (код для решения задачи)

```
import math
from fractions import Fraction
from itertools import combinations
import numpy as np
import matplotlib.pyplot as plt

def fr(x, max_den=10000):
    if abs(x - round(x)) < 1e-12:
        return str(int(round(x)))
    return str(Fraction(float(x)).limit_denominator(max_den))

def fmt(x, digits=6):
    return f"{x:.{digits}f}"

def vector_to_str(v, digits=6):
    return "[" + ", ".join(fmt(x, digits) for x in v) + "]"

def vector_to_fraction_str(v):
    return "[" + ", ".join(fr(x) for x in v) + "]"

def array_int_or_float_to_str(arr):
    out = []
    for x in arr:
        if abs(x - round(x)) < 1e-12:
            out.append(str(int(round(x))))
        else:
            out.append(fmt(x))
    return "[" + ", ".join(out) + "]"

def pure_strategy_analysis(C):
    row_mins = C.min(axis=1)
    col_maxs = C.max(axis=0)

    v_lower = row_mins.max()
    v_upper = col_maxs.min()

    saddle_points = []
    if np.isclose(v_lower, v_upper):
        good_rows = np.where(np.isclose(row_mins, v_lower))[0]
        good_cols = np.where(np.isclose(col_maxs, v_upper))[0]
        for i in good_rows:
            for j in good_cols:
                if np.isclose(C[i, j], v_lower):
                    saddle_points.append((i, j))

    return v_lower, v_upper, saddle_points

def analytic_solution_by_supports(C, tol=1e-9):
    m, n = C.shape
    solutions = []

    for k in range(1, min(m, n) + 1):
        for I in combinations(range(m), k):
            for J in combinations(range(n), k):
                sub = C[np.ix_(I, J)].astype(float)

                if abs(np.linalg.det(sub)) < tol:
                    continue

                inv_sub = np.linalg.inv(sub)
                ones = np.ones(k)

                denom = float(ones @ inv_sub @ ones)
                if abs(denom) < tol:
                    continue

                x_support = (ones @ inv_sub) / denom
                y_support = (inv_sub @ ones) / denom
                v = 1.0 / denom
```

```

        if np.any(x_support < -tol) or np.any(y_support < -tol):
            continue

        x = np.zeros(m)
        y = np.zeros(n)
        x[list(I)] = x_support
        y[list(J)] = y_support

        x[x < tol] = 0.0
        y[y < tol] = 0.0

        sx = x.sum()
        sy = y.sum()
        if sx <= tol or sy <= tol:
            continue

        x /= sx
        y /= sy

        row_payoffs = C @ y
        col_payoffs = x @ C

        if np.any(row_payoffs > v + tol):
            continue
        if np.any(col_payoffs < v - tol):
            continue
        if np.any(np.abs(row_payoffs[list(I)] - v) > tol):
            continue
        if np.any(np.abs(col_payoffs[list(J)] - v) > tol):
            continue

        solutions.append({
            "k": k,
            "rows_support": tuple(i + 1 for i in I),
            "cols_support": tuple(j + 1 for j in J),
            "x": x,
            "y": y,
            "v": v,
            "row_payoffs": row_payoffs,
            "col_payoffs": col_payoffs,
        })

    if not solutions:
        return None

    solutions.sort(key=lambda s: (-s["k"], s["rows_support"], s["cols_support"]))
    return solutions[0]

def brown_robinson(C, epsilon=0.1, max_iter=10000, start_row=0, start_col=0):
    m, n = C.shape

    row_counts = np.zeros(m, dtype=int)
    col_counts = np.zeros(n, dtype=int)

    row_cum = np.zeros(m, dtype=float)
    col_cum = np.zeros(n, dtype=float)

    i = start_row
    j = start_col

    best_upper = math.inf
    best_lower = -math.inf

    history = []

    for k in range(1, max_iter + 1):
        row_counts[i] += 1
        col_counts[j] += 1

        row_cum += C[:, j]
        col_cum += C[i, :]

        upper_curr = row_cum.max() / k
        lower_curr = col_cum.min() / k

```

```

best_upper = min(best_upper, upper_curr)
best_lower = max(best_lower, lower_curr)
gap = best_upper - best_lower

x_est = row_counts / k
y_est = col_counts / k

history.append({
    "k": k,
    "row_choice": i + 1,
    "col_choice": j + 1,
    "row_cum": row_cum.copy(),
    "col_cum": col_cum.copy(),
    "upper_curr": upper_curr,
    "lower_curr": lower_curr,
    "upper_best": best_upper,
    "lower_best": best_lower,
    "gap": gap,
    "x_est": x_est.copy(),
    "y_est": y_est.copy(),
    "row_counts": row_counts.copy(),
    "col_counts": col_counts.copy(),
})

if gap <= epsilon:
    break

i = int(np.argmax(row_cum))
j = int(np.argmin(col_cum))

return history

def print_rec(rec):
    row_cum_str = array_int_or_float_to_str(rec["row_cum"])
    col_cum_str = array_int_or_float_to_str(rec["col_cum"])
    print(
        f"{rec['k']:>4} | "
        f" x{rec['row_choice']:<2} | "
        f" y{rec['col_choice']:<2} | "
        f"{row_cum_str:<30} | "
        f"{col_cum_str:<30} | "
        f"{fmt(rec['upper_curr']):>10} | "
        f"{fmt(rec['lower_curr']):>10} | "
        f"{fmt(rec['upper_best']):>10} | "
        f"{fmt(rec['lower_best']):>10} | "
        f"{fmt(rec['gap']):>10}"
    )

def print_brown_robinson_table(history, cut=False):
    if not history:
        return

    print("Таблица метода Брауна-Робинсон")
    print("-" * 150)
    header = (
        f"{'k':>4} | {'x_i':>4} | {'y_i':>4} | "
        f"{'накопл. выигрыши A по строкам':<30} | "
        f"{'накопл. выигрыши A по столбцам':<30} | "
        f"{'v_max/k':>10} | {'v_min/k':>10} | {'min v_max':>10} | {'max v_min':>10} | {'E':>10}"
    )
    print(header)
    print("-" * 150)

    history_start = history[:7] if cut else history

    for rec in history_start:
        print_rec(rec)

    if cut:
        print(" " * 70 + "." * 10 + " " * 70)
        for rec in history[-7:]:
            print_rec(rec)

    print("-" * 150)
    print()

```

```

def plot_brown_robinson_graphs(history, analytic=None, save=False, prefix="lab2"):
    if not history:
        print("История метода Брауна-Робинсон пуста, графики построить нельзя.")
        return

    ks = np.array([rec["k"] for rec in history], dtype=int)

    upper_curr = np.array([rec["upper_curr"] for rec in history], dtype=float)
    lower_curr = np.array([rec["lower_curr"] for rec in history], dtype=float)
    upper_best = np.array([rec["upper_best"] for rec in history], dtype=float)
    lower_best = np.array([rec["lower_best"] for rec in history], dtype=float)
    gaps = np.array([rec["gap"] for rec in history], dtype=float)

    x_est_all = np.array([rec["x_est"] for rec in history], dtype=float) # shape = (K, m)
    y_est_all = np.array([rec["y_est"] for rec in history], dtype=float) # shape = (K, n)

    m = x_est_all.shape[1]
    n = y_est_all.shape[1]

    plt.figure(figsize=(10, 6))
    plt.plot(ks, upper_best, label="Лучшая верхняя оценка")
    plt.plot(ks, lower_best, label="Лучшая нижняя оценка")
    plt.plot(ks, upper_curr, linestyle="--", alpha=0.7, label="Текущая верхняя оценка")
    plt.plot(ks, lower_curr, linestyle="--", alpha=0.7, label="Текущая нижняя оценка")

    if analytic is not None:
        plt.axhline(analytic["v"], linestyle=":", linewidth=2,
                    label=f"Аналитическая цена игры v = {analytic['v']:.6f}")

    plt.xlabel("Номер итерации k")
    plt.ylabel("Оценка цены игры")
    plt.title("Сходимость оценок цены игры в методе Брауна-Робинсон")
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.tight_layout()

    if save:
        plt.savefig(f"{prefix}_value_bounds.png", dpi=300, bbox_inches="tight")

    plt.figure(figsize=(10, 6))
    for i in range(m):
        plt.plot(ks, x_est_all[:, i], label=f"x{i + 1}")

    if analytic is not None:
        for i in range(m):
            plt.axhline(
                analytic["x"][i],
                linestyle=":",
                linewidth=1,
                alpha=0.9,
                label=f"x{i + 1} * = {analytic['x'][i]:.6f}" if ks[0] == 1 else None
            )

    plt.clf()
    plt.figure(figsize=(10, 6))
    for i in range(m):
        plt.plot(ks, x_est_all[:, i], label=f"x{i + 1} (числ.)")
        plt.axhline(analytic["x"][i], linestyle=":", linewidth=1, alpha=0.9, label=f"x{i + 1} * (аналит.)")

    plt.xlabel("Номер итерации k")
    plt.ylabel("Частота использования стратегии")
    plt.title("Сходимость смешанной стратегии игрока A")
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.tight_layout()

    if save:
        plt.savefig(f"{prefix}_player_A_frequencies.png", dpi=300, bbox_inches="tight")

    plt.figure(figsize=(10, 6))
    for j in range(n):
        plt.plot(ks, y_est_all[:, j], label=f"y{j + 1}")

    if analytic is not None:

```

```

        for j in range(n):
            plt.axhline(
                analytic["y"][j],
                linestyle=":",
                linewidth=1,
                alpha=0.9,
                label=f"y{j + 1}* = {analytic['y'][j]:.6f}" if ks[0] == 1 else None
            )

    plt.clf()
    plt.figure(figsize=(10, 6))
    for j in range(n):
        plt.plot(ks, y_est_all[:, j], label=f"y{j + 1} (числ.)")
        plt.axhline(analytic["y"][j], linestyle=":", linewidth=1, alpha=0.9, label=f"y{j +
1}* (аналит.)")

    plt.xlabel("Номер итерации k")
    plt.ylabel("Частота использования стратегии")
    plt.title("Сходимость смешанной стратегии игрока B")
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.tight_layout()

    if save:
        plt.savefig(f"{prefix} player B frequencies.png", dpi=300, bbox_inches="tight")

    plt.figure(figsize=(10, 6))
    plt.plot(ks, gaps, label="E(k) = min(v_max/k) - max(v_min/k)")
    plt.xlabel("Номер итерации k")
    plt.ylabel("Погрешность")
    plt.title("Изменение погрешности метода Брауна-Робинсон")
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.tight_layout()

    if save:
        plt.savefig(f"{prefix} gap.png", dpi=300, bbox_inches="tight")

    plt.show()

def main():
    C = np.array([
        [9, 10, 13],
        [1, 18, 11],
        [17, 4, 0],
    ], dtype=float)

    print()
    print("Матрица игры A:")
    for row in C:
        print(' '.join(f'{int(x):>{4}}' for x in row))
    print()

    v_lower, v_upper, saddles = pure_strategy_analysis(C)

    print("Анализ игры в чистых стратегиях")
    print(f"Нижняя цена игры max(min): {fmt(v_lower)}")
    print(f"Верхняя цена игры min(max): {fmt(v_upper)}")

    if saddles:
        print("Седловые точки есть:")
        for i, j in saddles:
            print(f"  A{i + 1}, B{j + 1}, значение = {fmt(C[i, j])}")
    else:
        print("Седловой точки нет => решение ищем в смешанных стратегиях.")
    print()

    # Аналитическое решение
    print("Аналитическое решение")
    analytic = analytic_solution_by_supports(C)

    if analytic is None:
        print("Аналитическое решение этим способом не найдено.")
        print()
    else:

```

```

print(f"Найдена опора игрока A: {analytic['rows_support']}")
print(f"Найдена опора игрока B: {analytic['cols_support']}")
print(f"Цена игры  $v = \{fmt(analytic['v'])\} = \{fr(analytic['v'])\}$ ")
print(f"Оптимальная стратегия A:  $x^* = \{vector\_to\_str(analytic['x'])\}$ ")
print(f"Оптимальная стратегия A (дроби): {vector_to_fraction_str(analytic['x'])}")
print(f"Оптимальная стратегия B:  $y^* = \{vector\_to\_str(analytic['y'])\}$ ")
print(f"Оптимальная стратегия B (дроби): {vector_to_fraction_str(analytic['y'])}")
print()
print("Проверка:")
print(f"Выигрыши строк против  $y^*$ : {vector_to_str(analytic['row_payoffs'])}")
print(f"Выигрыши столбцов против  $x^*$ : {vector_to_str(analytic['col_payoffs'])}")
print("Опорные должны быть равны  $v$ ")
print()

# Браун-Робинсон
print("Метод Брауна-Робинсон")
print(f"Точность  $\epsilon = 0.1$ ")
print(f"Начальная пара стратегий: A1, B1")
print()

history = brown_robinson(C, epsilon=0.1, max_iter=10000, start_row=0, start_col=0, )
print_brown_robinson_table(history, cut=True)

last = history[-1]
k = last["k"]
x_br = last["x_est"]
y_br = last["y_est"]
v_top = last["upper_best"]
v_bottom = last["lower_best"]
v_mid = (v_top + v_bottom) / 2

print("Итоги метода Брауна-Робинсон")
print(f"Число итераций: {k}")
print(f"Лучшая верхняя оценка цены игры: {fmt(v_top)} = {fr(v_top)}")
print(f"Лучшая нижняя оценка цены игры: {fmt(v_bottom)} = {fr(v_bottom)}")
print(f"Погрешность  $E = \{fmt(last['gap'])\}$ ")
print(f"Приближённая цена игры (середина интервала): {fmt(v_mid)} = {fr(v_mid)}")
print()

print("Число использований стратегий:")
print(f"A: {last['row_counts']}")
print(f"B: {last['col_counts']}")
print()

print("Частоты использования стратегий (приближение смешанных стратегий):")
print(f"A:  $x \sim = \{vector\_to\_str(x\_br)\}$ ")
print(f"A:  $x \sim$  (дроби) = {vector_to_fraction_str(x_br)}")
print(f"B:  $y \sim = \{vector\_to\_str(y\_br)\}$ ")
print(f"B:  $y \sim$  (дроби) = {vector_to_fraction_str(y_br)}")
print()

# Сравнение
print("Сравнение методов")
if analytic is not None:
    print(f"Аналитическая цена игры: {fmt(analytic['v'])}")
    print(f"Браун-Робинсон, средняя оценка: {fmt(v_mid)}")
    print(f"Разница: {fmt(abs(analytic['v'] - v_mid))}")
    print()
    print("Сравнение стратегий:")
    print(f"Аналитическая  $x^*$ : {vector_to_str(analytic['x'])}")
    print(f"Браун-Робинсон  $x \sim$ : {vector_to_str(x_br)}")
    print(f"Аналитическая  $y^*$ : {vector_to_str(analytic['y'])}")
    print(f"Браун-Робинсон  $y \sim$ : {vector_to_str(y_br)}")
else:
    print("Сравнение по стратегии и цене с аналитическим решением не выполняется,")
    print("так как аналитическое решение не найдено.")
print()
plot_brown_robinson_graphs(history, analytic=analytic, save=True, prefix="lab2")

if __name__ == "__main__":
    main()

```